GUÍA DE REQUERIMIENTOS Y LINEAMIENTOS - HACKATHON DE ARQUITECTURA DE SISTEMAS (PLATAFORMA EDITORIAL MVC)

Documentación técnica y especificaciones funcionales para el desarrollo del proyecto final de Arquitectura de Sistemas.

1. Introducción y Contexto del Reto

El presente reto de arquitectura tiene como objetivo el diseño e implementación de una Plataforma Web Editorial Multicanal, orientada a la gestión, comercialización y cotización personalizada de fondos bibliográficos. La industria editorial contemporánea requiere soluciones informáticas capaces de procesar desde ventas minoristas tradicionales hasta complejas operaciones corporativas o institucionales (bibliotecas, universidades, redes de librerías).

Los equipos participantes deberán construir un sistema web robusto que contemple un catálogo bibliográfico dinámico, un motor de compras automatizado con reglas de descuento complejas y un módulo de atención personalizada mediante asesores comerciales.

El requerimiento arquitectónico central de este certamen es la aplicación rigurosa del **Patrón Arquitectónico Modelo-Vista-Controlador (MVC)**, garantizando un desacoplamiento efectivo entre la lógica de negocio, la capa de persistencia, la gestión de peticiones y las interfaces de usuario.

2. Organización de los Equipos y Roles

Cada equipo estará conformado estrictamente por **3 integrantes**. Para asegurar una ejecución disciplinada y emular un entorno de desarrollo profesional, se asignarán las siguientes responsabilidades operativas y técnicas:

2.1. Líder Técnico & Arquitecto MVC / Backend

- **Responsabilidad Principal:** Diseñar la arquitectura general del sistema, el esquema de la base de datos y la capa de datos (**Modelo**).
- **Funciones Clave:**

- Implementar las entidades del dominio, lógica de persistencia y mapeo objeto-relacional (ORM o consultas estructuradas).
- Definir e implementar la regla de negocio avanzada en el Modelo (motor de descuentos, cálculo de impuestos y subtotales).
- Velar por el cumplimiento de los principios de diseño de software y la integridad referencial de los datos.

2.2. Ingeniero de Software / Controlador e Integraciones

- **Responsabilidad Principal:** Desarrollar el flujo de la aplicación, endpoints, enrutamiento y orquestación (**Controlador**).
- **Funciones Clave:**
  - Construir los controladores para interceptar peticiones HTTP, validar payload de entrada y coordinar respuestas.
  - Diseñar e implementar los mecanismos de autenticación, control de acceso basado en roles (RBAC) y gestión de sesiones.
  - Garantizar la seguridad del software contra vulnerabilidades comunes (inyecciones SQL, XSS, CSRF).

2.3. Desarrollador Frontend & UX/QA

- **Responsabilidad Principal:** Diseñar la interfaz gráfica, experiencia de usuario (**Vista**) y ejecutar pruebas de calidad.
- **Funciones Clave:**
  - Crear vistas dinámicas, limpias y responsivas accesibles desde múltiples dispositivos.
  - Implementar scripts cliente para interactividad y validación de formularios en tiempo de entrada.
  - Elaborar el plan de pruebas funcionales y certificar los entregables antes del despliegue final.

3. Requerimientos Funcionales Detallados

RF-01: Landing Page y Catálogo Bibliográfico Interactivo

- **RF-01.1 Vista Principal:** Módulo de presentación con publicaciones destacadas, novedades del mes y banners promocionales.

- **RF-01.2 Búsqueda y Filtrado:** Buscador en tiempo real por título, autor, ISBN o editorial. Filtros combinables por género literario, rango de precio y formato (tapa blanda, tapa dura, e-book).
- **RF-01.3 Ficha Técnica del Libro:** Vista detallada de cada obra que incluye: portada, sinopsis, autor, año de edición, stock disponible, precio base y ficha bibliográfica completa.

RF-02: Gestión de Usuarios, Roles y Seguridad

El sistema debe implementar autenticación segura y autorizar operaciones según cinco roles estrictamente delimitados:

1. **Visitante (Anónimo):** Puede navegar por el catálogo, aplicar filtros y visualizar fichas técnicas. No puede procesar compras ni solicitar cotizaciones sin autenticarse.
2. **Cliente Minorista:** Usuario registrado con perfil de compra individual. Accede al carrito de compras estándar y al historial de sus pedidos.
3. **Cliente Institucional/Mayorista:** Cuenta validada para compras a volumen (colegios, universidades, empresas). Accede a escalas especiales de precio y módulo de cotizaciones.
4. **Asesor de Ventas:** Personal comercial encargado de revisar, ajustar y aprobar/rechazar solicitudes de pedidos personalizados o cotizaciones mayoristas.
5. **Administrador:** Acceso total a la plataforma para gestión de catálogo, usuarios, roles, métricas globales y parámetros del sistema.

RF-03: Motor de Pedidos y Carrito con Lógica de Negocio en el Modelo

- **RF-03.1 Gestión de Carrito:** Adición, modificación de cantidades y eliminación de ítems. Persistencia de estado según sesión de usuario.
- **RF-03.2 Reglas Dinámicas de Descuento (Enfocadas en el Modelo):** La lógica de cálculo DEBE ejecutarse exclusivamente dentro de la capa del Modelo, aplicando la siguiente matriz de reglas:
    - **Descuento por Volumen (Unidades por título o combinación):**
        - 1 a 5 unidades: 0% de descuento.
        - 6 a 20 unidades: 5% de descuento sobre el subtotal.
        - 21 a 50 unidades: 10% de descuento sobre el subtotal.
        - Más de 50 unidades: 15% de descuento sobre el subtotal.
    - **Descuento Adicional por Tipo de Cliente:**
        - Cliente Minorista: 0% adicional.
        - Cliente Institucional / Mayorista: 8% de descuento adicional aplicable acumulativamente tras el descuento por volumen.



- **RF-03.3 Cierre de Orden:** Generación de orden de compra con desglose explícito: Subtotal Base, Descuento Aplicado (con detalle de regla), Impuestos y Total Final.

RF-04: Módulo de Solicitud de Pedidos Personalizados y Cotizaciones

- **RF-04.1 Solicitud Especial:** Permitir a Clientes Institucionales solicitar ediciones especiales, impresiones bajo demanda o volúmenes masivos no disponibles en el stock actual.
- **RF-04.2 Formulario de Requerimientos:** Captura de variables: título/obra, cantidad estimada, acabado de portada, tipo de papel y fecha requerida de entrega.
- **RF-04.3 Flujo de Trabajo Comercial:** La solicitud se asigna a la bandeja de un Asesor de Ventas, quien puede emitir una cotización formal en formato digital para aprobación del cliente.

RF-05: Panel de Administración y Control Comercial

- **RF-05.1 CRUD de Catálogo:** Creación, lectura, actualización y eliminación lógica de libros, categorías, autores y stock disponible.
- **RF-05.2 Monitoreo de Órdenes:** Visualización de pedidos procesados, filtrados por estado (*Pendiente*, *Aprobado*, *Enviado*, *Cancelado*).
- **RF-05.3 Gestión de Cotizaciones:** Panel para que el Asesor de Ventas evalúe y responda a las solicitudes del módulo RF-04.

4. Requerimientos no funcionales

4.1. Separación Estricta de Capas

- **Modelo (Model):** Contiene la lógica de negocio, reglas de cálculo, clases de entidad y métodos de acceso a datos. **Queda estrictamente prohibido incluir código HTML, scripts de interfaz o lectura directa de peticiones HTTP en esta capa.**
- **Controlador (Controller):** Recibe las solicitudes del cliente, invoca la lógica de los Modelos adecuados y selecciona la Vista a renderizar. **No debe contener lógica de negocio ni sentencias SQL explícitas.**
- **Vista (View):** Presenta los datos al usuario final. Debe estar desacoplada de la base de datos; solo consume variables e información pasadas explícitamente por el Controlador.

4.2. Seguridad

- **Protección de Credenciales:** Almacenamiento de contraseñas obligatoriamente procesadas mediante algoritmos de hashing seguros (Bcrypt, Argon2 o PBKDF2).
- **Validación de Entradas:** Sanitización y validación estricta de parámetros en lado servidor para prevenir inyecciones SQL, ataques XSS (Cross-Site Scripting) y CSRF (Cross-Site Request Forgery).
- **Autorización:** Verificación de permisos y roles a nivel de Controlador antes de ejecutar cualquier acción sensible o renderizar vistas privadas.

4.3. Usabilidad, Rendimiento y Diseño Responsivo

- **Interfaz de Usuario:** Diseño adaptable mediante CSS Grid/Flexbox o frameworks como Bootstrap/Tailwind. Optimizado para navegación fluida en dispositivos móviles, tablets y escritorio.
- **Tiempo de Respuesta:** Tiempos de carga de página inferiores a 2 segundos en condiciones locales normales.

4.4. Control de Versiones

- Repositorio Git estructurado con commits claros y descriptivos. Uso de ramas funcionales (*feature branches*) para el desarrollo paralelo de cada integrante.

5. Entregables Esperados

Al finalizar el tiempo límite del Hackathon, cada equipo deberá entregar:

1. **Código Fuente:** Repositorio público de Git (GitHub/GitLab) con el código completo del proyecto.
2. **Base de Datos:** Script SQL exportado (.sql) con el esquema relacional DDL e información semilla inicial DML para pruebas (mínimo 10 libros, 5 usuarios con distintos roles y categorización previa).
3. **Documento:** Archivo en la raíz del proyecto que contenga:
    - Diagrama de Arquitectura de Sistemas y estructura de carpetas MVC.
    - Guía de instalación y ejecución paso a paso en entorno local.
    - Credenciales de acceso de prueba para los 5 roles definidos.
4. **Demostración en Vivo:** Presentación técnica de 10 minutos ante el jurado evaluador demostrando el flujo completo del sistema y defendiendo la arquitectura utilizada.



6. Rúbrica Detallada de Calificación (Sobre 100 Puntos)

Criterio	Descripción / Indicadores	Puntos Máx.
Arquitectura MVC y Limpieza de Código	Separación clara y estricta entre Modelo, Vista y Controlador. Mantenibilidad, modularidad, ausencia de acoplamiento inadecuado (ej. código SQL en Vistas o HTML en Modelos).	**25**
Lógica de Negocio y Motor de Descuentos	Implementación correcta y precisa de las reglas dinámicas de descuento (por volumen y por tipo de cliente) ejecutadas de forma encapsulada en el Modelo.	**20**
Gestión de Roles y Seguridad	Autenticación funcional, control de acceso por roles (RBAC) para las 5 entidades, hashing de contraseñas y sanitización de entradas contra vulnerabilidades.	**20**
Funcionalidad y Completitud del Sistema	Cumplimiento completo de los requerimientos funcionales (Catálogo, Carrito, Cotizaciones, CRUD y Panel de Administración).	**15**
Diseño Frontend, UX y Adaptabilidad	Interfaz atractiva, moderna, intutitiva y plenamente responsiva en diferentes tamaños de pantalla.	**10**
Uso de Git, Documentación y Sustentación	Repositorio organizado con buena frecuencia de commits por integrante, calidad del `README.md`, claridad y fluidez en la demostración en vivo.	**10**
TOTAL	**Evaluación global del proyecto**	**100**